Nam Tu - 153076622 - Exercise 1

In [1]:
```
%matplotlib inline
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt


df = pd.read_csv("./samples.csv", quotechar="'")


df = df.iloc[1:].copy()

df["Elapsed time"] = pd.to_timedelta(
    "00:" + df["Elapsed time"].astype(str)
).dt.total_seconds()

df["ECG I"] = pd.to_numeric(df["ECG I"])
df["ECG I filtered"] = pd.to_numeric(df["ECG I filtered"])
```

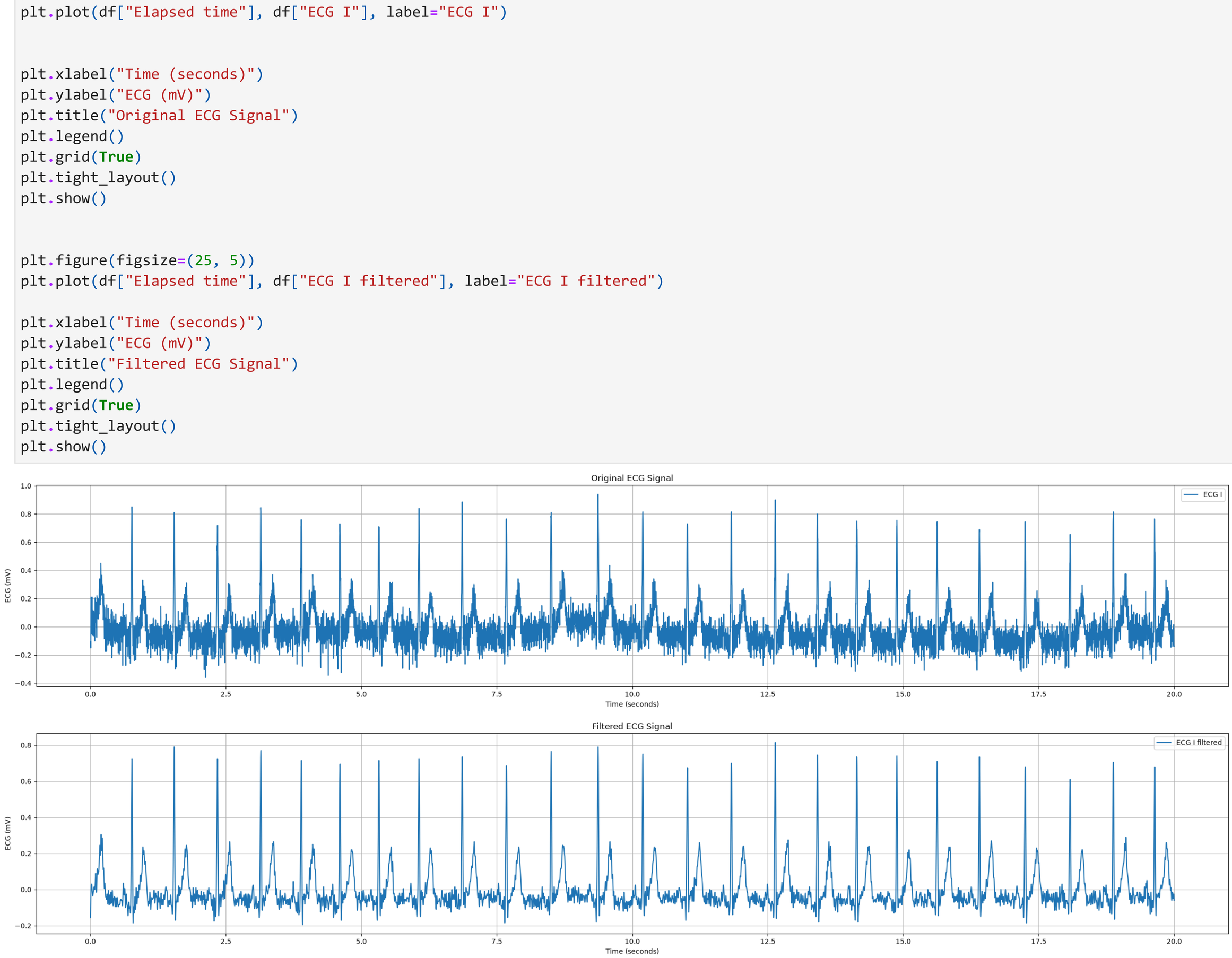
1. The shape of the ECG signal can be more clearly seen after filtering than the original, and the peaks are significantly less sharp after filtering. A common possible artefact that is filtered is the 50Hz interference from the local powerline. There could also be minor noise from other sources such as the interference from local devices.

In [85]:
```
# Plot both signals
plt.figure(figsize=(25, 5))
plt.plot(df["Elapsed time"], df["ECG I"], label="ECG I")


plt.xlabel("Time (seconds)")
plt.ylabel("ECG (mV)")
plt.title("Original ECG Signal")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()


plt.figure(figsize=(25, 5))
plt.plot(df["Elapsed time"], df["ECG I filtered"], label="ECG I filtered")


plt.xlabel("Time (seconds)")
plt.ylabel("ECG (mV)")
plt.title("Filtered ECG Signal")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



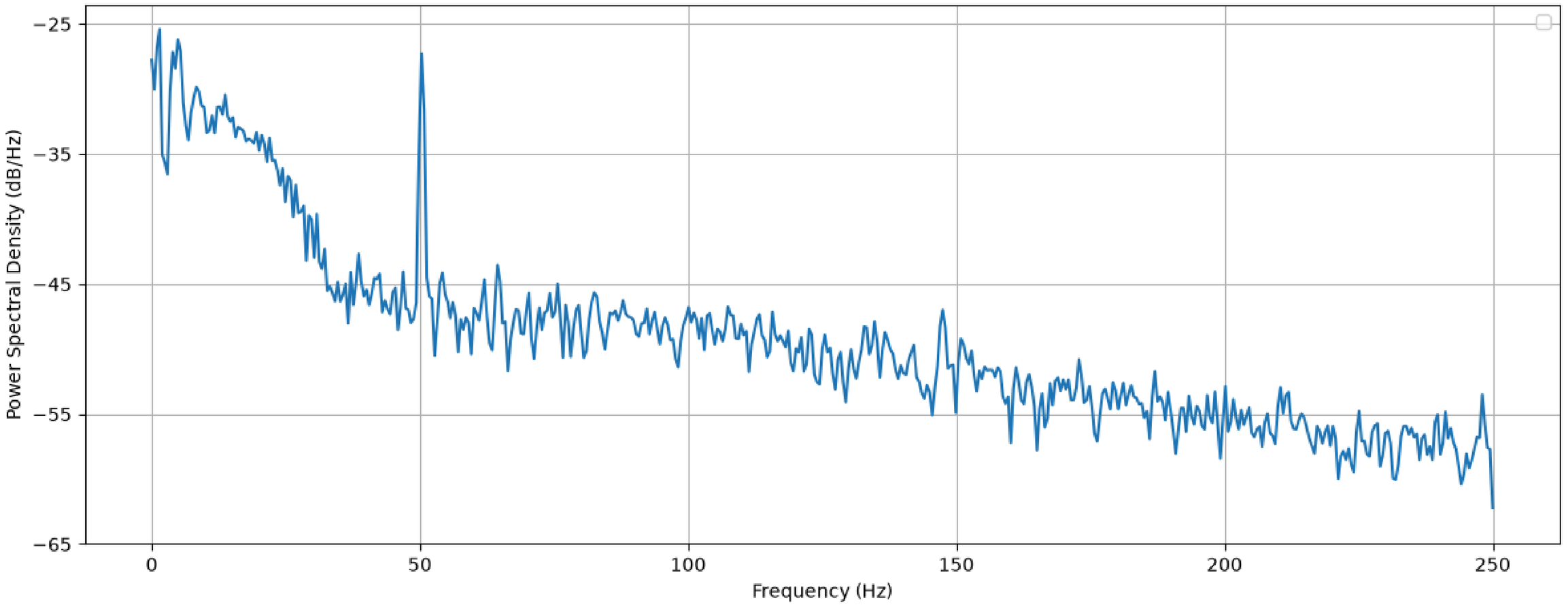
2. The PSD shows a large peak at the 50Hz mark in the original ECG signal's PSD, which is later not present after filtering. This is the 50Hz powerline inteference being filtered out. Additionally, the filtering mostly kept frequencies from 0-50Hz, i.e. a low-pass filter was applied for the 0-50Hz range, which is precisely the range of frequencies of a typical ECG signal.

In [38]:
```
plt.figure(figsize=(12, 5))
plt.psd( df["ECG I"],Fs= 500, NFFT=1024)
plt.xlabel("Frequency (Hz)")
```

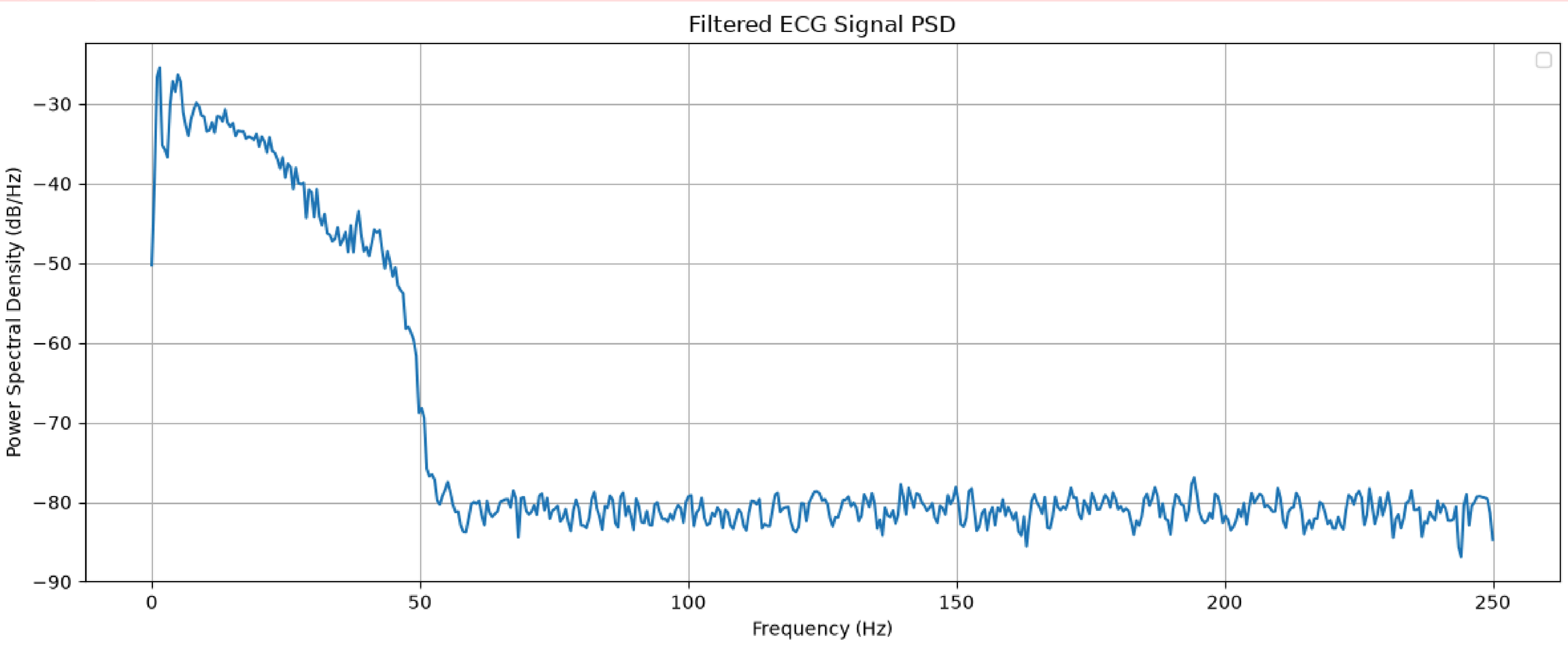
```
plt.title("Original ECG Signal PSD")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()


plt.figure(figsize=(12, 5))
plt.psd( df["ECG I filtered"],Fs=500, NFFT=1024)
plt.xlabel("Frequency (Hz)")
plt.title("Filtered ECG Signal PSD")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
C:\Users\haina\AppData\Local\Temp\ipykernel_7652\2653200279.py:5: UserWarning: No artists with labels found to put in legend.  Note that artists whose label start with an underscore are ignored when legend() is called with no argument.
  plt.legend()
```




```
C:\Users\haina\AppData\Local\Temp\ipykernel_7652\2653200279.py:15: UserWarning: No artists with labels found to put in legend.  Note that artists whose label start with an underscore are ignored when legend() is called with no argument.
  plt.legend()
```



3. Limited signal to the first 5000 samples for better visualization.

The algorithm can be seen to miss some R peaks and consequently the remaining peaks in a cycle, while it identifies all correctly for the filtered signal. Perhaps the small and rapid fluctations in the noise makes the sharp rise for the R peak undetectable for the algorithm. Therfore, while they are identifiable to the eye, perhaps it is computationally unclear for the algorithm if it is looking for a sharp slope for the R peak.

However, once the algorithm has correctly identified an R peak in the unfiltered signal, the remaining peaks are identified correctly with high precision (the identifying lines match perfectly between unfiltered and filtered ECG). Thus, the hardest peak to identify for the algorithm is the R peak, the rest falls into place when it has been identified.

In [30]:
```
import matplotlib.pyplot as plt
import neurokit2 as nk


sampling_rate = 500
for ecg in ["ECG I", "ECG I filtered"]:
    ecg_cleaned = df[ecg][0:5000]

    _, rpeaks = nk.ecg_peaks(ecg_cleaned, sampling_rate=sampling_rate)


    _, waves_peak = nk.ecg_delineate(
        ecg_cleaned,
        rpeaks,
        sampling_rate=sampling_rate,
    )

    # Extract onset indices for each wave type
    r_indices = rpeaks["ECG_R_Peaks"]
    p_indices = [x for x in waves_peak["ECG_P_Peaks"] if not np.isnan(x)]
    q_indices = [x for x in waves_peak["ECG_Q_Peaks"] if not np.isnan(x)]
    s_indices = [x for x in waves_peak["ECG_S_Peaks"] if not np.isnan(x)]
    t_indices = [x for x in waves_peak["ECG_T_Peaks"] if not np.isnan(x)]

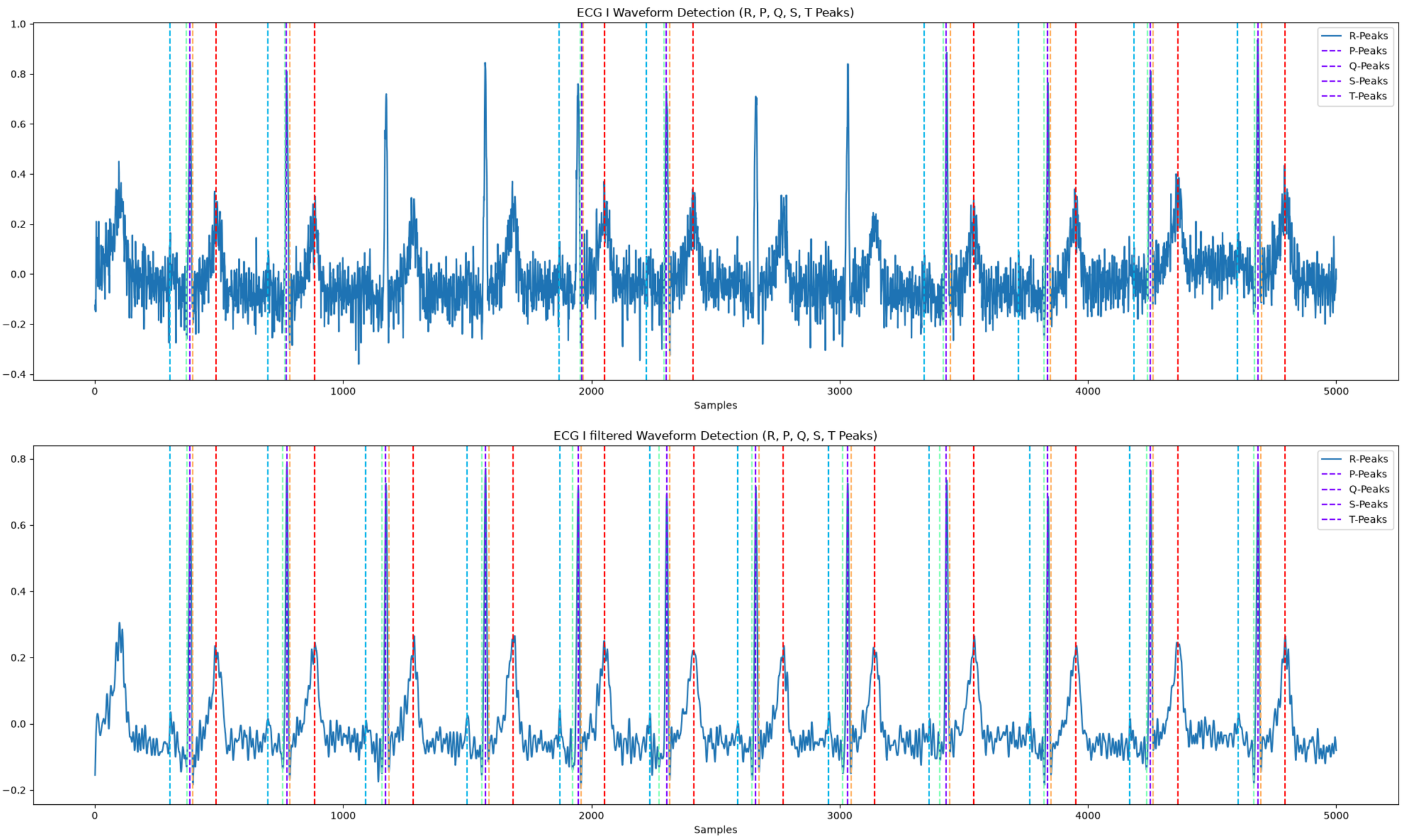

    all_waveforms = [r_indices, p_indices, q_indices, s_indices, t_indices]

    fig = nk.events_plot(all_waveforms, signal=ecg_cleaned)
    fig.set_size_inches(20, 6, forward=True)

    plt.title(f"{ecg} Waveform Detection (R, P, Q, S, T Peaks)")
    plt.xlabel("Samples")


    plt.gca().legend(["R-Peaks", "P-Peaks", "Q-Peaks", "S-Peaks", "T-Peaks"])

    plt.tight_layout()
    plt.show()
```



In [34]:
```
signals, info = nk.ecg_process(df['ECG I'], sampling_rate=1/0.002)

rpeaks = info["ECG_R_Peaks"]

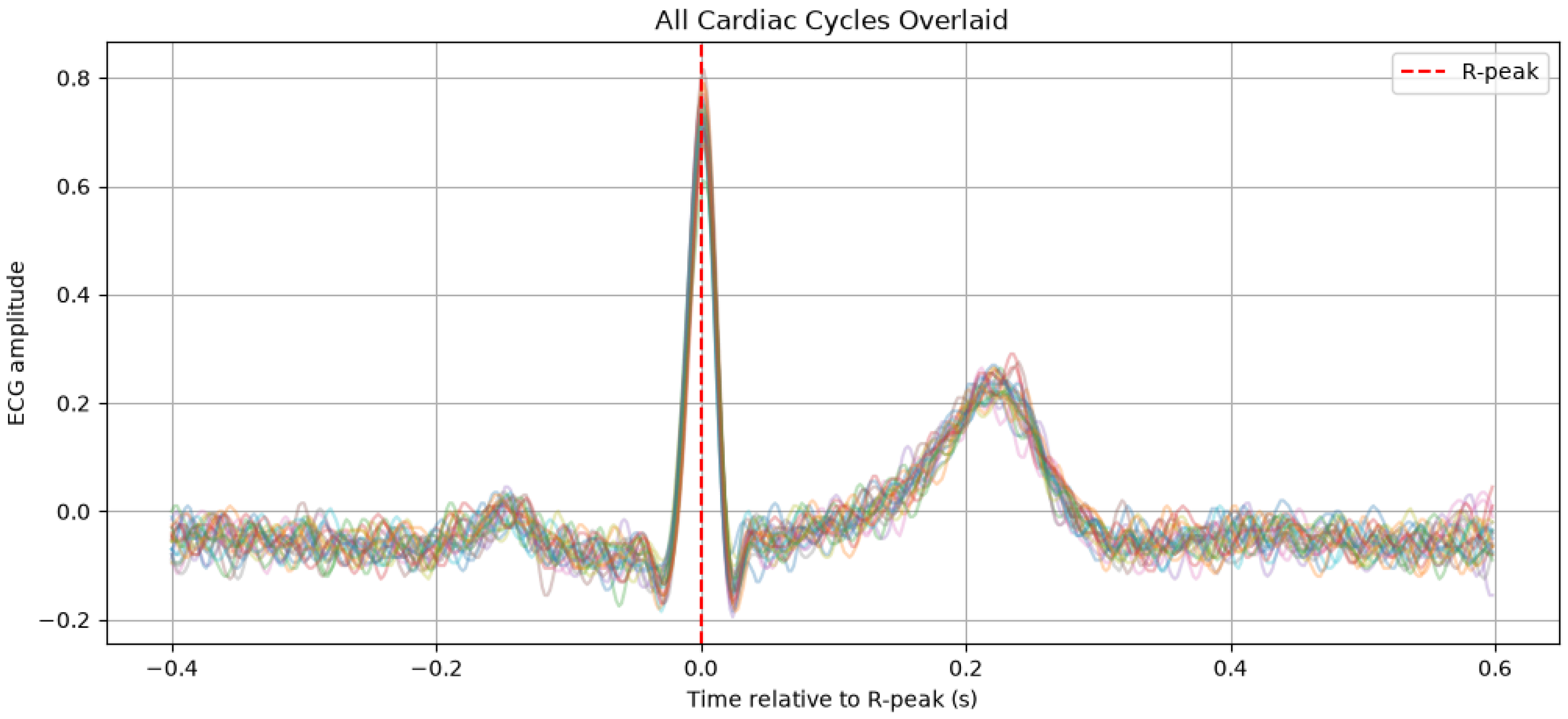
fs = 500
before = int(0.4 * fs)
after = int(0.6 * fs)

plt.figure(figsize=(12, 5))

for r in rpeaks:
    if r - before >= 0 and r + after < len(df):
        cycle = df["ECG I filtered"].iloc[r-before:r+after].to_numpy()
        time = np.arange(-before, after) / fs
        plt.plot(time, cycle, alpha=0.3)

plt.axvline(0, color="red", linestyle="--", label="R-peak")
```

```
plt.xlabel("Time relative to R-peak (s)")
plt.ylabel("ECG amplitude")
plt.title("All Cardiac Cycles Overlaid")
plt.grid(True)
plt.legend()
plt.show()
```



4. The low-pass filter is for filtering out the unwanted frequencies above 50Hz, because the 0-50Hz is the range of the ECG signal. The notch filter at 50Hz specific is to filter out the interference of the local powerline because that is its frequency. The signal after being passed through the filters are more clear ("less fuzzy") than compared to the original signal, but it is not as clear as the already-given filtered signal at the beginning.

Perhaps more filtering techniques were used to obtain such a signal such as a stricter filter, and for example, a moving average window to "filter out" the unecessary small peaks and keep the most apparent peaks of the ECG. Filtering targeting other rhytmic and/or electrical processes in our body is also possible such as breathing (0.1 Hz to 0.3 Hz), and the electrical signals from the muscles (50 Hz to 150 Hz), etc.

In [27]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import butter, buttord, sosfiltfilt, filtfilt, iirnotch

ecg_signal = df['ECG I']



fs = 500
wp = 150
ws = 151
gpass = 3.0
gstop = 15
N, Wn = buttord(wp, ws, gpass, gstop, fs=fs)
sos = butter(N=N, Wn=Wn, btype='lowpass', fs=fs, output='sos')
lowpass_signal = sosfiltfilt(sos, ecg_signal)

b,a = iirnotch(50/(fs/2),30)
notch_signal = filtfilt(b, a, lowpass_signal)
```

In [28]:
```
plt.figure(figsize=(20, 20))

plt.subplot(6, 1, 1)
plt.plot(df['Elapsed time'],ecg_signal,)
plt.title('Unfiltered ECG Signal')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude (mV)')

plt.subplot(6, 1, 2)
plt.psd( ecg_signal,Fs= 500, NFFT=1024)
plt.xlabel("Frequency (Hz)")
plt.title("Unfiltered ECG Signal PSD")



plt.subplot(6, 1, 3)
plt.plot(df['Elapsed time'],lowpass_signal)
plt.title('Low-pass filtered ECG Signal')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude (mV)')
```

```
plt.subplot(6, 1, 4)
plt.psd( lowpass_signal,Fs= 500, NFFT=1024)
plt.xlabel("Frequency (Hz)")
plt.title("Low-pass filtered ECG Signal PSD")



plt.subplot(6, 1, 5)
plt.plot(df['Elapsed time'],notch_signal)
plt.title('Low-pass filtered and Notch-filtered ECG Signal')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude (mV)')

plt.subplot(6, 1, 6)
plt.psd( notch_signal,Fs= 500, NFFT=1024)
plt.xlabel("Frequency (Hz)")
plt.title("Low-pass filtered Notch-filtered ECG Signal PSD")

plt.tight_layout()
```

